

Understanding Harness Engineering

From Agent Loops and Tool Interfaces to
Context, Sandboxes, Verification, and Long-Running Work

A short technical handbook for AI engineers, developers, and serious learners

@techNmak

ORIENTATION

What this handbook will make clear

A capable language model is not, by itself, a working agent.

Something has to decide what the model sees, expose the actions it can take, execute those actions, return observations, preserve useful state, enforce permissions, recover after failure, decide whether another step is needed, and determine whether the task actually succeeded.

That surrounding machinery is increasingly described as an **agent harness**.

The terminology is still young. Microsoft currently uses *agent harness* for runtime scaffolding that turns a language model into an agent by coordinating model calls, tool calls, context, state, approvals, and multi-step execution [15]. Anthropic uses a closely related runtime meaning when it separates a durable session, a harness that runs the model/tool loop, and a sandbox that executes code [12]. OpenAI uses *harness engineering* more broadly for the engineering of environments, feedback loops, repository structure, constraints, observability, and other scaffolding required for reliable agentic work [10].

For this handbook, we will use the following working distinction:

- **Agent harness:** the runtime and control machinery immediately surrounding a model and allowing it to act over multiple steps.
- **Harness engineering:** the broader engineering discipline of designing the interfaces, context policy, state, execution environment, controls, feedback systems, recovery behavior, and verification mechanisms that make agentic work reliable.

This is an editorial taxonomy for this handbook, not an industry-standard definition.

By the end, you should be able to explain:

- why a model call is not yet an agent;
- how a minimal agent loop works;
- how the model, harness, session, tools, and sandbox differ;
- why tool design changes agent behavior;
- why MCP is useful but does not specify a complete agent architecture;
- why active context is different from durable state;
- why compaction is lossy;
- how permissions, containment, credentials, and approvals solve different problems;
- why tool failures need model-usable error semantics;
- why retries become dangerous around non-idempotent side effects;
- why verification should be separated from self-reported completion;
- how long-running agents inherit familiar durability and consistency problems;
- when subagents help and when coordination costs dominate;
- how evaluation measures the model and the harness together;
- why harness assumptions should be retested as models improve.

Prerequisites: basic familiarity with language-model inference, function/tool calling, context windows, and ordinary software systems is enough.

Scope: this is a handbook about the systems layer around agentic model execution. It is not a complete MCP handbook, agent-security textbook, distributed-systems textbook, or catalog of every agent framework.

1 - A MODEL CALL IS NOT AN AGENT

A language-model invocation can be written abstractly as

$$y \sim p_{\theta}(\cdot \mid c)$$

where c is the context supplied to the model and y is the generated output.

Nothing in that expression automatically means that a shell command runs, a browser opens, a file changes, an API call is retried, a failed attempt is observed, or work continues after the first response.

An agentic system adds an external control process around model inference.

A useful conceptual decomposition is:

agent system $\sim$ model + harness + environment

This is not a formal identity. It is a mental model.

The **model** supplies learned capability. The **harness** decides how inference is connected to actions and repeated over time. The **environment** contains the external state that actions inspect or modify.

Anthropic distinguishes ordinary workflows, whose execution paths are substantially specified by application code, from agents, where the model dynamically chooses actions and strategies [4]. Microsoft similarly distinguishes ordinary agents, workflows, and a harness agent assembled for longer multi-step work [15].

The important boundary is therefore not “chatbot versus agent” as a marketing label. It is whether model output participates in a controlled interaction loop with external state.

2 - THE MINIMAL AGENT LOOP

A harness can be described with four evolving objects.

Let:

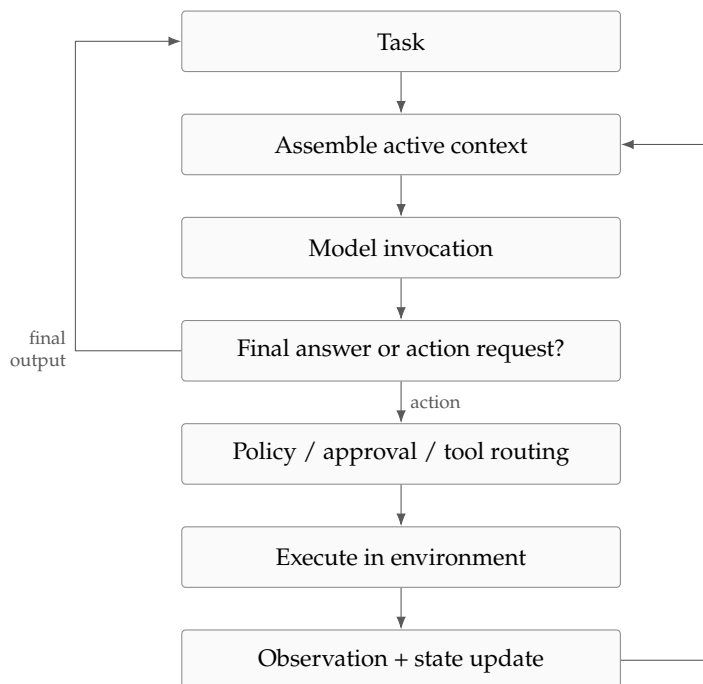
- s_t = durable or working state before step t
- $c_t = G(s_t)$ = the context selected for the next model call
- $a_t = M(c_t)$ = the model output, which may include an action request
- $o_t = E(a_t)$ = the observation produced by the environment if an action executes

The harness then updates state:

$$s_{(t+1)} = U(s_t, a_t, o_t)$$

This formalization is ours; it is not a standard API.

The loop is:



A modern runner usually performs some variation of this loop: call the model, inspect the response, return if it is final, execute tools if requested, append results, then call the model again [14].

ReAct is an important research predecessor. It studied language-model trajectories in which reasoning and task actions are interleaved so that external observations can affect later decisions [1]. Modern harnesses do not need to expose or persist private reasoning traces to retain the durable systems idea: **act, observe, update, continue**.

3 - WHAT THE HARNESS ACTUALLY CONTROLS

The word “harness” becomes useful when we list the responsibilities that do not belong to the model weights.

A harness may decide:

- which instructions and state enter the next context;
- which tools are visible;
- how tool schemas are represented;
- whether a tool call is permitted;
- whether human approval is required;
- how the tool executes;
- how tool results are transformed before returning to the model;
- how history is persisted;
- how context is compacted;
- when subagents are created;
- how errors are returned;
- how retries are handled;
- what budgets terminate a run;
- what evidence is required before accepting completion;
- how traces and metrics are recorded;
- how execution resumes after interruption.

Not every harness owns all of these concerns. Some systems delegate state, execution, policy, or recovery to other components.

That is why “harness” is best treated as a control-plane concept rather than a synonym for one library or process.

OpenAI’s current Agents API, for example, manages sessions, orchestration, context compaction, and recovery while allowing the application to choose tools and an execution environment [14]. Microsoft’s Harness composes chat clients, context providers, persistence, compaction, approvals, observability, and bounded looping [15].

These examples are implementation snapshots, not permanent definitions. As of this handbook’s publication, OpenAI’s Agents API is in public beta. Microsoft’s packaged Harness factory is released, while background agents, file access, and looping are still marked experimental, and its shell tooling is pre-release.

4 - MODEL, HARNESS, SESSION, TOOL, AND SANDBOX

These terms describe different system objects.

Term	Useful meaning
Model	The learned inference system
Agent	The model operating within an action loop toward a task
Harness	Runtime/control machinery driving that loop
Session	Logical history/state container for ongoing work; durability depends on the implementation
Context	Information supplied to a particular model invocation
Tool	A callable capability exposed to the model
Sandbox	A controlled execution environment
Workflow	An execution structure largely specified by code
Evaluation harness	Infrastructure that runs and grades evaluations

Anthropic's Managed Agents architecture explicitly separates session, harness, and sandbox. The session is an append-only log, the harness invokes the model and routes tool calls, and the sandbox runs code and edits files [12].

OpenAI makes a related distinction in its sandbox architecture: the harness is the **control plane** around model calls, routing, approvals, tracing, recovery, and run state; the sandbox is the **execution plane** that reads and writes files, runs commands, installs dependencies, and snapshots state [14].

The boundaries can be implemented differently. The conceptual separation remains useful because the components fail differently and carry different trust assumptions.

5 - AN AGENT-COMPUTER INTERFACE

A model never interacts with “the computer” in the abstract.

It interacts with an interface.

One coding harness might expose:

```
read_file(path, start_line, end_line)
search_code(query)
apply_patch(path, patch)
run_tests(target)
```

Another might expose only:

```
shell(command)
```

Both may eventually manipulate the same repository. They do not present the same problem to the model.

SWE-agent formalized this idea as the **Agent-Computer Interface (ACI)**. Instead of assuming that interfaces built for human developers are automatically optimal for language models, the work treats interface design itself as an experimental variable. The authors designed repository-navigation, editing, testing, and feedback mechanisms around agent behavior, then measured how those choices affected performance [2].

Their original paper reported 12.5% pass@1 on SWE-bench and 87.7% on HumanEvalFix for the evaluated SWE-agent setup [2]. These numbers are historical results for the paper’s models, harness, tasks, and evaluation procedure. They are not universal claims about ACI design.

The lasting contribution is the engineering question:

Is the capability merely available, or is it exposed in a form the model can use reliably?

6 - AFFORDANCE, CONTRACT, OBSERVATION, FEEDBACK

A useful way to analyze any agent-facing tool is through four properties.

Affordance. What operation can the tool perform?

Contract. How does the model request the operation?

Observation. What information comes back?

Feedback. Does the result make the next correct action easier?

Consider file editing.

A raw shell gives enormous affordance. It also forces the model to select commands, quote paths correctly, infer tool output, and recover from formatting mistakes.

A structured patch tool narrows the action surface but may make intent easier to express and outcomes easier to verify.

Neither interface is universally superior. The correct design depends on the model, task, environment, and evaluation.

The key point is that interface structure becomes part of the effective difficulty of the task [2,5].

7 - TOOLS ARE PART OF THE MODEL INPUT

A tool is not only executable code.

Its name, description, arguments, and result representation are usually placed into the model's context.

That means tool design affects inference before a tool is ever called.

Anthropic frames tool design as an interface problem between conventional software and probabilistic agent behavior. Their engineering work emphasizes clear purpose, unambiguous parameters, namespacing, useful response content, and token-efficient results [5].

Two costs are useful to separate:

$C_{\text{tool}} = C_{\text{selection}} + C_{\text{observation}}$

This is a conceptual decomposition.

$C_{\text{selection}}$ is the difficulty of choosing the correct tool and arguments.

$C_{\text{observation}}$ is the context and reasoning burden imposed by the returned result.

A tool with an elegant backend API can still be poor for an agent if its descriptions overlap with other tools or its results dump irrelevant data into the context window.

8 - MORE TOOLS CAN MAKE THE DECISION HARDER

A system might expose six nearly overlapping operations:

```
search()
find()
lookup()
query()
grep()
retrieve()
```

From a traditional capability perspective, this looks rich.

From the model's perspective, every additional tool adds another choice to the action space.

Anthropic reports a common failure mode in which bloated or overlapping tool sets create ambiguous decision points. Their tool-design guidance recommends selecting a smaller number of distinct operations where possible, using namespacing to separate related capabilities, and evaluating naming schemes rather than assuming one universal convention [5].

This gives a useful production rule:

Tool count is not a monotonic measure of agent capability.

Adding a tool can improve reachable behavior while simultaneously making tool selection more difficult.

That trade-off should be measured on representative tasks rather than settled by aesthetics.

9 - TOOL RESULTS SHOULD RETURN HIGH-SIGNAL CONTEXT

Suppose an agent asks for a customer record.

A low-level API might return hundreds of fields, internal identifiers, pagination metadata, image variants, audit records, and storage keys.

A human-written program can cheaply ignore fields.

A language model must consume returned text through its context budget.

Anthropic recommends returning high-signal information, using filters and pagination for potentially large results, and exposing concise versus detailed modes where appropriate [5,6]. The exact best format can vary by task and model.

This means a tool result should be judged by more than correctness.

Ask:

- Did it return the information needed for the next decision?
- Did it omit irrelevant payload?
- Did it preserve identifiers required for later actions?
- Did truncation explain how to retrieve more?
- Did the format make the result easy to interpret?

Tool output is part of context engineering.

10 - TOOL ERRORS ARE OBSERVATIONS TOO

A tool failure can terminate the run, trigger an automatic retry, or return a model-visible error and allow recovery.

Those semantics are different.

Imagine:

```
deploy_service(environment="prod")
```

fails because the requested branch does not exist.

An opaque result such as:

```
Error.
```

forces the model to guess.

A more useful result might provide:

```
status: failed
reason: branch_not_found
requested_branch: release/2.4
available_branches: [...]
side_effects: none
retryable: true
```

The exact schema is application-specific. The general principle is not.

OpenAI's current Agents SDK distinguishes two cases. Setting `toolNotFoundBehavior` to `return_error_to_model` can turn an unresolved function-tool call into a model-visible error so the run continues; separately, `toolErrorFormatter` customizes tool error messages returned to the model [14]. Anthropic likewise recommends actionable validation errors over opaque codes or raw tracebacks [5].

A recoverable failure should give the model enough information to choose a better next action without pretending the failed operation succeeded.

11 - MCP IS AN INTEROPERABILITY LAYER, NOT A COMPLETE AGENT LOOP

The Model Context Protocol standardizes interfaces between AI applications and external capabilities.

The dated MCP core server specification centers server primitives such as prompts, resources, and tools. Tasks is a separate opt-in MCP extension, not part of the core protocol. At publication time, first-party MCP surfaces describe its maturity inconsistently: the MCP extensions overview lists MCP Tasks among official extension repositories, while the ext-tasks repository README labels the repository experimental and not official, and the published Tasks specification page is marked Draft. Because those first-party labels conflict, this handbook treats Tasks as an evolving optional extension rather than assigning it a single maturity status [13].

The current published Tasks draft allows a server to return an asynchronous task handle for a `tools/call` and defines `tasks/get`, `tasks/update`, and `tasks/cancel`. It advises clients to persist task IDs to durable storage so polling can resume after a crash or restart. Task support is negotiated explicitly, and host support can vary [13].

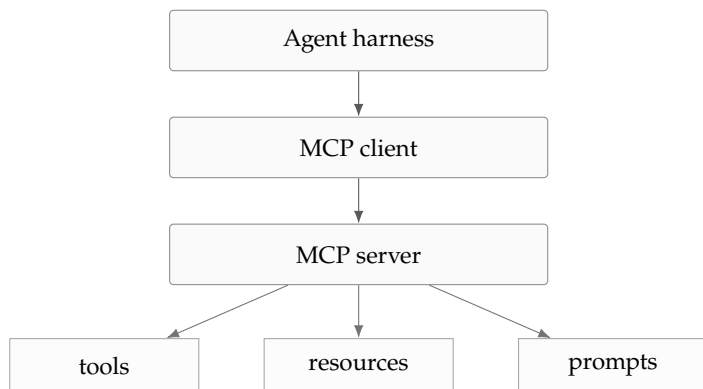
This makes MCP increasingly capable, but an important boundary remains.

MCP does not, by itself, determine:

- how the model loop is scheduled;
- how active context is selected;
- how a task is judged complete;
- which actions require approval;
- how side effects are retried;
- how a sandbox is provisioned;
- how multiple agents coordinate;
- how the overall agent is evaluated.

A harness may use MCP heavily while still owning those policies elsewhere.

Conceptually:



Optional extensions can add other protocol capabilities, such as asynchronous Tasks.

The safe mental model is **interoperable capability interface**, not **entire agent architecture**.

The current Tasks draft also illustrates why protocol state and tool outcome should not be collapsed. A tool result with `isError: true` still completes the task at the protocol level, while JSON-RPC execution errors use the task's failed state [13]. That distinction is useful when designing model-visible error handling and recovery.

12 - ACTIVE CONTEXT IS A HARNESS DECISION

Every model call sees a finite context.

The harness determines what enters it.

Potential ingredients include:

- system and developer instructions;
- the current user task;
- recent message history;
- tool schemas;
- tool results;
- retrieved documentation;
- plans and todo state;
- summaries of earlier work;
- file or repository state;
- memory records;
- outputs from subagents.

A naive policy is to include everything that might ever matter.

That eventually fails for two reasons.

First, context capacity is finite.

Second, relevance matters even when capacity is large. Irrelevant material competes with useful evidence for model attention and can make the decision problem harder.

Anthropic's context-engineering guidance therefore treats context as a constrained resource and emphasizes just-in-time retrieval, token-efficient tools, compaction, structured memory, and subagents [6].

13 - PROGRESSIVE DISCLOSURE INSTEAD OF MONOLITHIC INSTRUCTIONS

OpenAI's Codex-first engineering experiment encountered a practical documentation problem.

A large repository instruction file accumulated too much guidance. It consumed context, became difficult to maintain, and was hard to verify mechanically.

The team moved toward a shorter entry-point document that pointed agents to structured repository-local sources of truth [10].

The general pattern is **progressive disclosure**:

1. provide enough information to orient the agent;
2. make deeper information discoverable;
3. load detailed guidance only when the task requires it.

This is the same reason tool interfaces benefit from search, filters, and concise result modes.

Availability and relevance are different properties.

A large context window does not make an undifferentiated information dump a good architecture.

Agent Skills are another modern use of progressive disclosure. Anthropic and Microsoft describe skills as reusable packages of instructions and optional scripts/resources that can be discovered and loaded when relevant instead of placing all guidance in every prompt [25,26]. Unlike a tool, whose primary interface is a callable action, a skill primarily packages reusable guidance and supporting resources that the model can draw on while deciding how to work. Microsoft distinguishes this from an explicit workflow, where the execution path is defined by the application; its current guidance prefers workflows when checkpointing, predictable ordering, or non-idempotent side effects make replaying a flexible skill undesirable [26]. A 2026 preprint analyzes 307 **confirmed skill-induced failures** across SkillsBench and SWE-Skills-Bench, including 125 functional failures and 182 efficiency regressions [24]. That study attributes selected paired failures or cost regressions to loaded skills; it does **not** estimate a general base rate for how often skills fail across arbitrary deployments. Skills should therefore be evaluated rather than assumed beneficial. Because skills may contain instructions, resources, and executable scripts, their provenance is also a security concern; Section 22 returns to that boundary.

14 - DURABLE HISTORY AND ACTIVE CONTEXT ARE DIFFERENT STATE SURFACES

A long-running task can have more history than any one inference call should see.

This makes the distinction between **durable session state** and **active model context** fundamental.

Anthropic's Managed Agents design stores an append-only session log outside the active context window. The harness can retrieve slices of that history and transform them before passing selected information back to the model [12].

OpenAI's Agents API similarly separates a durable session from the context-management behavior performed by the managed harness [14].

The two state surfaces overlap, but neither generally contains the other. Durable history can remain outside the next model call, while active context can also contain ephemeral material such as the current user message, live tool schemas, or temporary runtime metadata that was never stored in the durable session.

The exact storage model differs by product. The principle is broader:

Persistent availability does not imply automatic inclusion in every inference call, and inclusion in one inference call does not imply durable storage.

15 - COMPACTION IS LOSSY

When a conversation grows too large, a common strategy is to replace older detail with a shorter summary. This reduces active-context cost.

It can also lose information.

Let H be a history and $C(H)$ a compacted representation. In general:

$$C(H) \neq H$$

The transformation cannot be assumed to preserve every detail future steps may need.

Anthropic explicitly warns that context-retention decisions can create failures because future turns may require information that seemed unimportant earlier. Their Managed Agents architecture therefore keeps durable session history separately from the compacted context seen by the model [6,12].

Compaction should be understood as **context transformation**, not equivalent to deleting durable history.

The engineering question is:

Which information can be summarized, which should remain directly recoverable, and how can the agent retrieve omitted detail later?

16 - CASE STUDY: HARNESS CHANGES CAN ALTER AGENT QUALITY

A useful real-world example came from Anthropic's April 2026 Claude Code postmortem [19].

Anthropic reported that its API and inference layer were unaffected, while three product-layer changes changed the quality users experienced in Claude Code and related products:

- the default reasoning-effort setting was reduced for some models and later reverted after user feedback;
- a session optimization intended to clear older reasoning after an idle period contained a bug that repeated the clearing behavior on later turns, making Claude appear forgetful and repetitive;
- a system-prompt instruction intended to reduce verbosity produced a 3% drop on one broader evaluation for both Opus 4.6 and Opus 4.7, and was reverted [19].

The point is not that these three mechanisms are universally important.

The point is architectural:

Observed agent quality can change when the product/harness configuration changes even when the underlying API and inference layer have not changed.

Prompt construction, reasoning settings, context retention, tool interfaces, and other runtime decisions are therefore part of the behavior of the deployed agent system.

17 - STRUCTURED WORKING STATE

Natural-language history is not the only way to preserve progress.

Long tasks often benefit from explicit artifacts such as:

- todo lists;
- issue trackers;
- feature status files;
- progress notes;
- test results;
- git commits;
- checklists;
- generated plans;
- output manifests.

Anthropic's long-running-agent experiments used an initializer session to prepare structured artifacts and later coding sessions to make incremental progress while leaving clear state for the next session [7]. Their example used feature lists, git history, progress notes, and an initialization script.

These artifacts are not hidden model memory.

They are external representations of task state.

That makes them inspectable, editable, versionable, and recoverable by future sessions or humans.

18 - EXECUTION ENVIRONMENTS

Some agents only read data.

Others run code, modify files, browse web applications, compile software, invoke internal systems, or create persistent artifacts.

Once an agent can cause side effects, the execution environment becomes part of the architecture.

A sandbox may provide:

- filesystem access;
- shell execution;
- installed packages;
- mounted data;
- network access;
- browser or computer control;
- ports;
- snapshots;
- resource limits.

OpenAI's sandbox documentation explicitly separates this execution plane from the harness control plane [14].

The environment determines both capability and failure mode.

A correct plan can fail because a package is missing. A poorly constrained environment can succeed while creating an unacceptable security risk.

Harness engineering therefore cannot stop at prompt design.

19 - A SANDBOX IS AN ISOLATION BOUNDARY, NOT THE ORCHES- TRATOR

A sandbox answers questions such as:

- Which files can this process reach?
- Which commands can it run?
- Is outbound networking allowed?
- What credentials or mounts are visible?
- What state survives the container?

A harness answers different questions:

- What should the model see next?
- Which tool call should execute?
- Does this call require approval?
- What result should be returned?
- Should the loop continue?
- What happens after failure?

The two can be colocated in a prototype.

They should still be reasoned about separately.

Keeping the control plane outside untrusted execution can protect session state, audit logs, credentials, and policy from model-generated code running inside the sandbox [12,14].

20 - APPROVAL AND CONTAINMENT SOLVE DIFFERENT PROBLEMS

Human approval can be valuable when an action has consequences that should remain under explicit human control.

Examples include deleting production data, sending external communication, transferring money, escalating privileges, or publishing changes.

But approval is not a complete security architecture.

Anthropic reports that users approved roughly 93% of Claude Code permission prompts in the telemetry discussed in its containment article [3]. That is Anthropic-specific product telemetry, not a universal approval rate. Their argument is that frequent prompts can create approval fatigue.

The same article gives a more extreme internal red-team example. In a controlled February 2026 exercise, a malicious prompt delivered through the user successfully exfiltrated credentials in 24 of 25 retries [3]. Anthropic's conclusion for that scenario was that environment-level filesystem and egress boundaries were the defense that held when model-layer intent checks could not distinguish the malicious instruction from an ordinary user request. This is a vendor-reported red-team result, not a general failure rate.

Containment addresses a different question:

What can the agent reach even if every higher-level behavioral safeguard fails?

Examples include filesystem boundaries, network egress rules, VMs, containers, and credential isolation.

Human review controls decisions.

Containment controls reachable capability.

21 - KEEP CREDENTIALS OUT OF UNTRUSTED EXECUTION

Suppose model-generated code runs inside a sandbox that also contains long-lived credentials.

Then a prompt injection, malicious dependency, or incorrect command may be able to read those secrets.

Anthropic describes a structural alternative in Managed Agents: credentials can live outside the execution sandbox and be used through controlled proxies or resource-specific mechanisms rather than being exposed directly to generated code [12].

Their containment work makes the general security principle clear [3]:

A resource that never enters the sandbox cannot be read directly from that sandbox.

This is a stronger boundary than telling the model not to access secrets.

One is an instruction.

The other changes the reachable state space.

22 - OBSERVATIONS HAVE PROVENANCE

Agent input does not only come from the user.

A harness may place into context:

- web pages;
- emails;
- issue descriptions;
- documents;
- files;
- database records;
- MCP resources;
- loaded skill instructions and resources;
- tool results;
- subagent reports;
- stored memory.

Those observations may contain hostile or misleading instructions.

A useful conceptual representation is:

`o_i = (content, source, trust, time, permissions)`

This is our mental model, not a standard wire format.

The point is that all strings are not equal merely because they become tokens.

A system instruction and text scraped from an arbitrary website can both enter model context while having very different provenance and authority.

Anthropic's containment work explicitly includes tools, files, and network access as external attack vectors and discusses persistent state as a new surface that can carry compromised information forward [3]. It also reports a concrete failure in which an approved network domain enabled data exfiltration through a legitimate API, leading Anthropic to treat an egress allowlist more like a **capability grant** than a simple destination filter. The same article warns that subagent output can create **trust escalation** if a parent agent treats it as inherently safer than raw tool output.

Independent research reaches the same broader security boundary from another direction. AgentDojo evaluates prompt injection against agents that operate over untrusted tool data; its **initial NeurIPS 2024 release** includes 97 realistic tasks and 629 security test cases across environments such as email, banking, travel, and messaging [21]. The benchmark exists precisely because data returned by tools can contain adversarial instructions that influence later agent actions.

Skills create a related trust boundary. Anthropic warns that skills can contain instructions and code and should be used only from trusted sources or audited carefully; a malicious skill can direct tool use or code execution in ways that do not match its stated purpose [25]. Microsoft likewise recommends treating skills like third-party code [26]. In its Agent Skills provider, `load_skill`, `read_skill_resource`, and `run_skill_script` require approval by default unless the application deliberately changes that policy for trusted sources.

23 - STOPPING CONDITIONS ARE PART OF THE HARNESS

An autonomous loop needs a rule for when to stop.

Possible stopping conditions include:

- the model returns a final output;
- a verifiable task predicate is satisfied;
- human input is required;
- a guardrail blocks further action;
- a tool enters a terminal failure;
- a resource budget is exhausted;
- a cancellation request arrives.

A production harness should distinguish **completion** from **termination**.

For example:

task complete $\neq$ turn budget exhausted

OpenAI's current Agents SDK supports explicit maximum-turn limits. Reaching that limit stops the loop but does not prove that the task succeeded [14].

Microsoft's Harness similarly exposes bounded-loop behavior and configurable iteration limits [15].

A stop can therefore mean success, policy block, timeout, budget exhaustion, failure, or human handoff.

Those states should not be collapsed into one generic "done."

24 - BUDGETS LIMIT AUTONOMY

A harness may enforce several independent budgets:

$B = (B_turns, B_tokens, B_time, B_cost, B_tools, B_concurrency)$

This vector is our conceptual notation.

Not every platform exposes each budget directly.

The design point is that autonomy is bounded by resources as well as permissions.

A run can be allowed to call a database but still be limited to 20 tool calls.

A subagent system can be permitted while limiting concurrent workers.

A browser agent can be allowed to operate for ten minutes but not indefinitely.

Budgets turn “keep trying” from an unbounded behavior into an operational policy [14,15].

They also make failures easier to classify. A task that times out is different from a task that reaches a verified success condition.

25 - VERIFICATION IS DIFFERENT FROM SELF-REPORTED COMPLETION

An agent can say:

The bug is fixed.

That is a claim.

It is not evidence.

For software work, stronger evidence might include:

- the original reproduction no longer fails;
- regression tests pass;
- the application was exercised end-to-end;
- the affected state changed as expected.

Anthropic's long-running-agent work found that coding agents could mark features complete without adequate end-to-end testing. Their harness improved behavior by requiring explicit verification with browser automation and structured feature status [7].

Independent evaluation work makes the same distinction concrete. In τ -bench, success is evaluated by comparing the final database state with an annotated goal state rather than trusting the agent's conversational claim. The benchmark also introduces pass^k to measure whether an agent succeeds consistently across repeated trials [22]. The conversational user in τ -bench is itself simulated by a language model, so the interaction trajectory is not deterministic even though the final environment state is compared with an annotated goal.

The general principle is:

Whenever practical, completion should be tied to observable state outside the model's own assertion.

For deterministic properties, deterministic verification is preferable.

For subjective properties, a separate evaluator or human may still be useful.

26 - DETERMINISTIC ENFORCEMENT VS MODEL JUDGMENT

Some decisions genuinely require interpretation:

- Is this search result relevant?
- Which file should be inspected next?
- Does this explanation satisfy the user's intent?

Other constraints can often be checked mechanically:

- Is this JSON valid?
- Is the amount above an allowed threshold?
- Is this path outside the mounted directory?
- Is this dependency edge forbidden?
- Has the run exceeded its tool-call budget?

A useful design rule is:

Use deterministic enforcement where the property is mechanically decidable. Use model or human judgment where interpretation is genuinely required.

This is a guideline, not a theorem.

OpenAI's current Agents SDK provides input, output, and tool guardrails that can validate or block execution [14]. Anthropic's containment work argues for environmental boundaries when the system can enforce hard reachability limits [3]. OpenAI's Codex-first repository similarly moved some architectural rules from prose into linters and structural tests [10].

The model should not have to remember a rule that the system can cheaply enforce.

27 - GENERATOR AND EVALUATOR ARE DIFFERENT ROLES

Some outputs cannot be fully checked by a deterministic test.

Examples include prose quality, design quality, research completeness, ambiguous planning, and certain forms of policy interpretation.

Anthropic experimented with separating generation from evaluation in long-running application work, using distinct roles to generate work, critique it, and iterate [11].

The useful concept is not that every task needs multiple agents.

It is that **producing an answer and judging an answer are different operations.**

An evaluator can still be wrong.

A model-based grader should therefore be treated as another fallible component whose own reliability needs evaluation.

Where deterministic checks exist, they remain stronger.

28 - FEEDBACK LOOPS GIVE THE AGENT EXTERNAL EVIDENCE

A one-shot model proposes output.

An agent can often do something more useful:

1. act;
2. observe the consequence;
3. revise.

For coding and operational work, useful feedback can include:

- compiler output;
- unit tests;
- integration tests;
- screenshots;
- browser state;
- logs;
- metrics;
- traces;
- database contents;
- API responses.

OpenAI's harness-engineering report describes making application state, logs, metrics, and traces accessible to Codex so the agent could inspect and verify its own changes [10].

A tool that changes state but cannot expose the resulting state creates a weak feedback loop.

Harness design is therefore partly about making relevant ground truth queryable.

29 - LONG-RUNNING AGENTS INHERIT DURABILITY PROBLEMS

A five-second tool call can often fail and be retried without much coordination.

A task that lasts hours or days is different.

During long-running work:

- the harness can crash;
- a worker can disappear;
- a sandbox can be reclaimed;
- the network can disconnect;
- a user can pause the task;
- approval can arrive hours later.

Google's Agent Executor is explicitly designed around this operational problem, with event logs, snapshots, resumption, and distributed execution for agents, harnesses, tools, and sandboxes [16]. As of this handbook's publication, Google's `google/ax` repository warns that its core concepts, protocols, and specifications are still being refined and may change substantially before a stable release.

Anthropic's Managed Agents architecture uses a durable session outside the harness so that a failed harness can be restarted and resume from the session log [12].

As agent work becomes longer and more distributed, harnesses inherit familiar systems concerns:

durability, recovery, consistency, retries, and failure isolation.

30 - RECOVERY REQUIRES CONTROL STATE AND WORLD STATE

Suppose a harness requests:

```
create_ticket(...)
```

The remote system creates the ticket.

Then the network connection drops before the harness receives the response.

After restart, the harness knows it intended to call the tool.

It may not know whether the external side effect committed.

Blindly running the same action again can create a duplicate ticket.

This produces an important distinction:

```
requested action  
executed action  
observed result  
committed task state
```

Those transitions do not necessarily fail atomically.

A durable event log helps the harness remember what it requested and observed [16].

It does not automatically make every external side effect safe to repeat [18].

31 - IDEMPOTENCY MATTERS FOR RETRIES

An operation is idempotent, in the usual systems sense, when repeating the same intended operation has the same intended effect as performing it once [18].

Abstractly:

$$f(f(s)) = f(s)$$

For example, an operation such as:

```
set_ticket_status(id, "closed")
```

can often be designed to be idempotent.

Operations such as:

```
charge_card(...)  
send_email(...)  
create_order(...)
```

are not naturally safe to repeat.

Reliable agent systems may therefore need mechanisms such as:

- idempotency keys;
- unique operation IDs;
- deduplication records;
- transactional writes;
- read-before-retry checks;
- tool-specific retry policies.

RFC 9110 uses the same underlying concept for idempotent HTTP request semantics and explains why idempotency matters when a client loses the response to a request.

The broader lesson is straightforward:

Automatic retry is a tool-specific policy, not a universal recovery strategy.

32 - CONCURRENCY CREATES CONSISTENCY PROBLEMS

Subagents and distributed workers can operate on shared state concurrently.

That raises familiar questions:

- In what order are updates committed?
- Can two workers overwrite one another?
- Can both claim the same task?
- Which event stream is authoritative?
- How are conflicting side effects detected?

Google's Agent Executor explicitly calls out session consistency and uses architectural constraints to reduce corruption in distributed agent workflows [16].

The exact solution depends on the application.

Some systems serialize writes.

Others partition state by task or resource.

The important point is that multi-agent concurrency is not only a prompting problem. It is also a state-consistency problem.

33 - SUBAGENTS ARE A HARNESS CHOICE

A harness may assign one model instance the whole task.

It may also delegate independent subtasks to additional agents.

Delegation can improve throughput when work decomposes cleanly.

It can also create:

- duplicated work;
- inconsistent assumptions;
- communication overhead;
- higher token cost;
- error propagation;
- conflict over shared state.

Google Research evaluated 180 agent configurations and found that architecture effects depended strongly on task structure. In their reported setup, centralized coordination improved Finance-Agent performance by 80.9% relative to the single-agent baseline, while every tested multi-agent variant degraded performance on the sequential PlanCraft task by 39-70% [17].

Those are results from that experimental setup, not universal scaling laws.

The durable rule is:

Delegation should be justified by task decomposition, not by a belief that more agents are inherently better.

34 - AGENT LEGIBILITY

OpenAI uses the phrase **agent legibility** for an important property of agent-first engineering environments [10].

The basic idea is to make the information an agent needs discoverable from the systems it can inspect.

For a code repository, that may include structured documentation, predictable package boundaries, clear ownership, test entry points, searchable logs, local metrics, and runnable application state.

For a data system, it may mean discoverable schemas and metadata.

For UI work, it may mean screenshots, DOM state, and deterministic ways to launch the application.

This is more than documentation quality.

A system can be easy for its original human authors to understand because they carry tribal knowledge that the agent does not have.

Harness engineering pushes important assumptions into inspectable artifacts.

35 - MECHANISMS ARE STRONGER THAN PROSE FOR ENFORCEABLE RULES

Suppose an architecture rule says:

Domain A must not import internal implementation code from Domain B.

One option is to write that rule in an instruction file.

Another is to implement a structural test that rejects the forbidden dependency.

The second changes the failure mode.

The agent no longer needs to remember perfectly. The invalid state fails verification.

OpenAI reports using custom linters and structural tests to enforce architectural rules in its Codex-first repository [10].

Instructions remain useful because not every property can be formalized.

But where a rule is objective and mechanically checkable, encoding it in code reduces dependence on model obedience.

36 - OBSERVABILITY HAS TWO AUDIENCES

Traditional observability helps engineers understand running software.

Agent systems add a second audience: the agent itself.

Useful signals for human operators include model turns, tool calls, tool failures, latency, token use, cost, retries, approval events, and final outcomes.

Useful signals for the agent can include application logs, traces, screenshots, browser state, test results, and resource state.

The first category helps people debug the harness.

The second gives the model evidence about the system it is working on.

OpenAI's Codex-first engineering work demonstrates this by exposing logs, metrics, and traces directly to coding agents [10].

37 - AGENT HARNESS VS EVALUATION HARNESS

An **agent harness** lets a model operate.

An **evaluation harness** measures how the resulting system performs.

Anthropic describes an evaluation harness as infrastructure that runs tasks end-to-end, provisions stable environments, records trials, executes graders, and aggregates results [8].

Independent benchmarks reinforce why that environment matters. τ -bench evaluates tool-using agents against final database state and repeated-trial reliability [22]. ToolSandbox evaluates stateful tool use with implicit dependencies between tools, on-policy conversations, and intermediate/final milestones over arbitrary trajectories [23]. Together they make it harder for an agent to receive credit merely for producing a plausible transcript.

That distinction matters because an agent evaluation typically measures more than the model.

A useful conceptual decomposition is:

measured agent performance = $F(\text{model}, \text{harness}, \text{environment}, \text{task}, \text{grader})$

This is not a statistical model; it is a reminder about dependency.

Changing the harness can change the result without changing the model weights.

Changing the environment can do the same.

38 - THE ENVIRONMENT CAN CHANGE THE BENCHMARK SCORE

Static question-answer benchmarks mostly score model outputs.

Agentic coding benchmarks can involve package installation, compilation, test execution, process spawning, disk use, CPU and memory pressure, and repeated tool interaction.

That makes infrastructure part of the task.

Anthropic experimentally varied compute resources on Terminal-Bench 2.0 and reported a six-percentage-point difference between its least- and most-resourced configurations in its experiment ($p < 0.01$). The shape of the result matters. From 1x through roughly 3x resource headroom, infrastructure failures fell substantially while success scores remained within statistical noise ($p = 0.40$). Above roughly 3x, additional resources began enabling strategies that materially changed task success. Anthropic therefore recommends skepticism toward leaderboard gaps below three percentage points when resource methodology is not documented and matched [9].

The correct conclusion is not:

More RAM makes the model smarter.

It is:

An agentic benchmark can measure interactions between model behavior and runtime resources.

Meaningful comparison therefore requires enough detail about the harness and environment to reproduce the operating conditions.

39 - HARNESS ASSUMPTIONS CAN EXPIRE

A harness often contains workarounds for known model behavior.

Those workarounds should not be assumed permanent.

Anthropic reports that context-reset machinery added because Claude Sonnet 4.5 could prematurely wrap up work near its context limit became unnecessary when the same harness was used with Claude Opus 4.5. The behavior had changed, so the reset became dead weight. In the same set of experiments, Anthropic reports a specific RetroForge comparison in which a solo Opus 4.5 run lasted 20 minutes and cost \$9, while the fuller harness ran for six hours and cost \$200. The author reports that the fuller harness produced substantially better output quality in that experiment, but it was more than 20x as expensive. Later experiments with Opus 4.6 removed additional scaffolding, including the sprint decomposition. The planner remained useful, while the evaluator became more task-dependent: it added the most value when the task sat near the model's capability edge. These are task-specific experiments, not universal cost ratios [11].

OpenAI's current Agents API likewise treats the Codex harness as a versioned system that evolves alongside model capabilities [20].

This suggests a stronger design principle:

Every harness component is a hypothesis about a current limitation, and it should justify both its quality lift and its execution cost for the model and task distribution being served.

When the model changes, test that hypothesis again.

Otherwise harnesses accumulate obsolete layers that add latency, token cost, complexity, and new failure modes.

40 - COMMON FAILURE MODES

A production harness should make failures observable and classifiable.

Tool-selection failure

The capability exists, but the model chooses the wrong tool.

Tool-contract failure

The intended tool is correct, but arguments are invalid.

Observation failure

The action succeeds, but the returned information is noisy, ambiguous, or incomplete.

Context failure

Important information is omitted, stale, buried, or destroyed during compaction.

State failure

The system loses track of progress, approvals, plans, or prior actions.

Environment failure

Packages, files, browser state, compute resources, or permissions differ from assumptions.

Verification failure

The model declares success without testing the external result.

Authorization failure

The agent is either blocked from legitimate work or granted more authority than needed.

Recovery failure

A retry duplicates a side effect or a restart cannot reconcile control state with world state.

Coordination failure

Subagents duplicate work, diverge in assumptions, or spend more resources coordinating than solving the task.

Budget failure

The run consumes its allowed turns, time, tokens, cost, or concurrency before reaching a successful terminal state.

Provenance / prompt-injection failure

Untrusted tool data, retrieved content, stored state, skills, or subagent output is treated as authoritative instruction and redirects the agent [3,21].

Skill-induced failure

A loaded skill is topically relevant but causes required work to be implemented incorrectly or omitted, or adds enough unnecessary procedure to create a material efficiency regression [24].

Harness engineering is largely the work of turning these broad failure categories into observable, testable system properties.

41 - COMMON MISCONCEPTIONS

“The harness is the system prompt.”

No. Instructions are one part of a harness. The harness can also own state, tool dispatch, approvals, compaction, recovery, budgets, and verification.

“The harness is the sandbox.”

No. The harness controls work; the sandbox is one place where work executes [14].

“MCP is the agent.”

No. MCP standardizes interoperable capability surfaces and extensions. The agent loop and its policies still require orchestration [13].

“Context window equals memory.”

No. Durable state can exist outside the current model context.

“Compaction preserves the full conversation.”

No. A summary is a lossy representation unless omitted information remains recoverable elsewhere.

“More tools always improve capability.”

No. Overlap and noisy results can make selection and downstream reasoning harder.

“A human approval prompt makes arbitrary execution safe.”

No. Approval and environmental containment solve different problems.

“If a tool failed, retry it.”

Not automatically. Side-effecting operations may need idempotency or deduplication semantics.

“The agent said it finished, so the task is complete.”

Self-report is weaker than observable verification.

“More agents are better.”

Not generally. Coordination cost can dominate on tightly sequential work [17].

“A better model makes every old harness improvement useful.”

No. Scaffolding can become unnecessary as model behavior changes [11].

“Skills are just harmless prompt templates.”

No. Skills can contain instructions, resources, and executable scripts. Third-party skills should be treated as provenance-bearing code/configuration and reviewed, sandboxed, and approval-gated according to the capabilities they expose [25,26].

42 - A PRODUCTION MENTAL MODEL

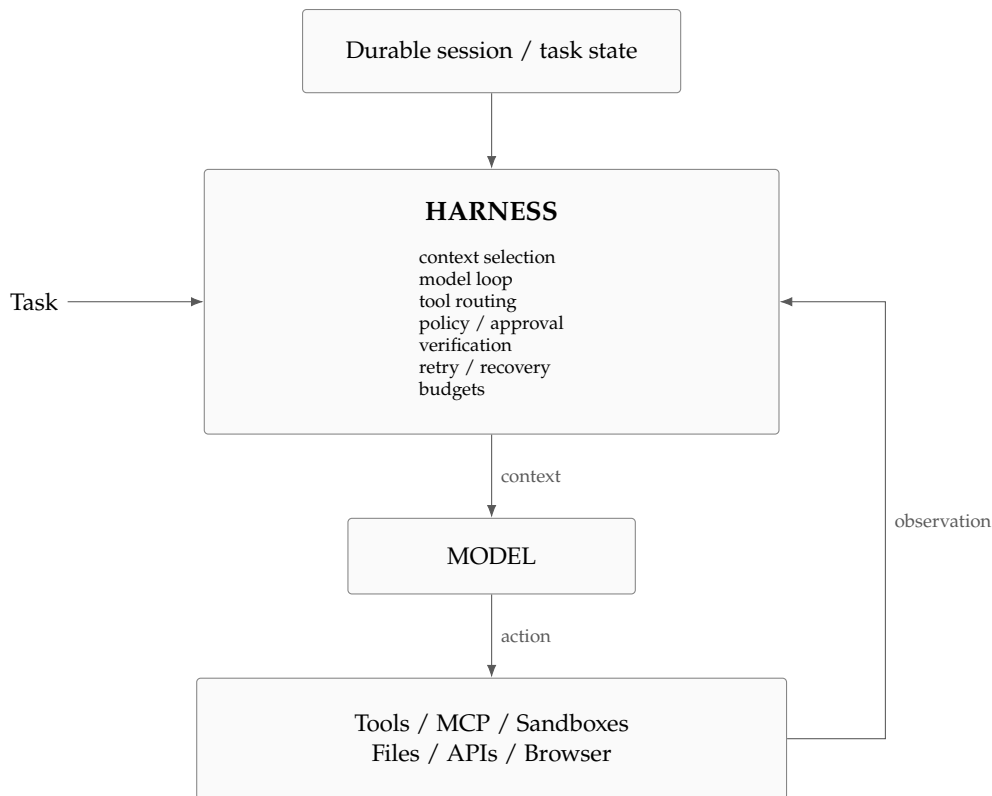
When evaluating an agent system, ask:

1. **What does the model actually receive?**
Instructions, history, tool schemas, retrieved state, summaries, artifacts.
2. **Who controls the loop?**
Application code, a managed harness, a workflow engine, or another agent.
3. **What actions are available?**
Function tools, MCP tools, shell, browser, files, APIs, subagents.
4. **Where do actions execute?**
Trusted process, local machine, container, VM, remote service.
5. **What survives the next turn or process crash?**
Session events, files, snapshots, external side effects.
6. **Which rules are enforced mechanically?**
Schemas, mounts, network boundaries, approvals, tests, budgets.
7. **How are errors represented?**
Fatal exception, retry, model-visible observation, human escalation.
8. **Which actions are safe to retry?**
Idempotent operations, deduplicated operations, or neither.
9. **How does the agent obtain ground truth?**
Tests, logs, browser state, database state, external responses.
10. **What proves completion?**
A model claim, deterministic validator, external state, evaluator, or human.
11. **How does the run stop if it never succeeds?**
Turn, time, token, tool, cost, or policy budgets.
12. **What happens after interruption?**
Resume, retry, reprovision, rollback, or restart.
13. **Can untrusted observations carry instructions?**
Web data, tool output, files, memory, loaded skills, subagent reports.
14. **How are subagents coordinated?**
Task boundaries, shared state, merge policy, concurrency limits.
15. **How is the harness evaluated?**
Ablations, representative tasks, stable environments, outcome metrics.

A system that cannot answer these questions may still work in a demo. It is much harder to reason about in production.

43 - THE COMPLETE MENTAL MODEL

A compact production picture is:



Hard boundaries: permissions · credentials · filesystem · network · schemas · budgets

Feedback: tests · UI state · logs · metrics · traces · outcomes · graders

The model supplies learned capability.

The harness decides how that capability is connected to work.

The persistent session/state layer determines what can survive across turns or restarts.

The context policy determines what the model can see now.

The execution environment determines what can actually happen.

Permissions and containment determine what failure can reach.

Feedback tells the system whether an action worked.

Verification determines whether “done” is evidence-backed.

Recovery determines whether work can continue after interruption.

Evaluation determines whether the harness is helping rather than merely adding machinery.

That is the core of harness engineering.

REFERENCES

Numbered citations in the text point to the sources below. Current product and specification behavior was checked in October 2026.

- [1] S. Yao et al. *ReAct: Synergizing Reasoning and Acting in Language Models*. ICLR 2023.
<https://arxiv.org/abs/2210.03629>
- [2] J. Yang et al. *SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering*. NeurIPS 2024.
<https://arxiv.org/abs/2405.15793>
- [3] Anthropic. *How we contain Claude across products*. 2026.
<https://www.anthropic.com/engineering/how-we-contain-claude>
- [4] Anthropic. *Building effective agents*.
<https://www.anthropic.com/engineering/building-effective-agents>
- [5] Anthropic. *Writing effective tools for agents — with agents*. 2025.
<https://www.anthropic.com/engineering/writing-tools-for-agents>
- [6] Anthropic. *Effective context engineering for AI agents*.
<https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>
- [7] Anthropic. *Effective harnesses for long-running agents*. 2025.
<https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents>
- [8] Anthropic. *Demystifying evals for AI agents*. 2026.
<https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents>
- [9] Anthropic. *Quantifying infrastructure noise in agentic coding evals*. 2026.
<https://www.anthropic.com/engineering/infrastructure-noise>
- [10] OpenAI. *Harness engineering: leveraging Codex in an agent-first world*. 2026.
<https://openai.com/index/harness-engineering/>
- [11] Anthropic. *Harness design for long-running application development*. 2026.
<https://www.anthropic.com/engineering/harness-design-long-running-apps>
- [12] Anthropic. *Scaling Managed Agents: Decoupling the brain from the hands*. 2026.
<https://www.anthropic.com/engineering/managed-agents>
- [13] Model Context Protocol. *Core server specification, Extensions overview, and Tasks extension*. Current documentation consulted October 2026.
<https://modelcontextprotocol.io/specification/2026-07-28/server>
<https://modelcontextprotocol.io/extensions/overview>
<https://github.com/modelcontextprotocol/ext-tasks>
<https://tasks.extensions.modelcontextprotocol.io/specification/draft/tasks>
- [14] OpenAI. *Agents API, Agents SDK, and Sandbox Agents documentation*. Current documentation consulted October 2026.
<https://developers.openai.com/api/docs/guides/agents-api/overview>
<https://developers.openai.com/api/docs/guides/agents/sandboxes>
<https://openai.github.io/openai-agents-js/guides/running-agents/>
<https://openai.github.io/openai-agents-js/guides/guardrails/>
- [15] Microsoft. *Agent Framework: Agent Harness*. Current documentation consulted October 2026.
<https://learn.microsoft.com/agent-framework/agents/harness>
- [16] Google Cloud. *Introducing Agent Executor, Google’s distributed Agent Runtime*. 2026.
<https://cloud.google.com/blog/products/ai-machine-learning/agent-executor-googles-distributed-agent-runtime/>
- [17] Google Research. *Towards a science of scaling agent systems: When and why agent systems work*. 2026.
<https://research.google/blog/towards-a-science-of-scaling-agent-systems-when-and-why-agent-systems-work/>
- [18] IETF. *RFC 9110: HTTP Semantics, Section 9.2.2 Idempotent Methods*. 2022.
<https://www.rfc-editor.org/rfc/rfc9110.html>
- [19] Anthropic. *An update on recent Claude Code quality reports*. 2026.
<https://www.anthropic.com/engineering/april-23-postmortem>

- [20] OpenAI. *Introducing the Agents API*. 2026.
<https://openai.com/index/introducing-the-agents-api/>
- [21] E. Debenedetti et al. *AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents*. NeurIPS 2024, Datasets and Benchmarks Track.
<https://doi.org/10.52202/079017-2636>
- [22] S. Yao, N. Shinn, P. Razavi, K. Narasimhan. *τ -bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains*. ICLR 2025.
<https://arxiv.org/abs/2406.12045>
- [23] J. Lu et al. *ToolSandbox: A Stateful, Conversational, Interactive Evaluation Benchmark for LLM Tool Use Capabilities*. Findings of NAACL 2025.
<https://aclanthology.org/2025.findings-naacl.65/>
- [24] G. Dong et al. *Agent Skills Can Be Harmful: An Empirical Study of Skill-Induced Failures in LLM Agents*. arXiv:2608.11888, 2026.
<https://arxiv.org/abs/2608.11888>
- [25] Anthropic. *Agent Skills*. Claude Platform documentation. Current documentation consulted October 2026.
<https://platform.claude.com/docs/en/agents-and-tools/agent-skills/overview>
- [26] Microsoft. *Agent Skills*. Agent Framework documentation. Current documentation consulted October 2026.
<https://learn.microsoft.com/agent-framework/agents/skills>